

Software Defect Prediction with Naïve Bayes Classifier

Aqsa Rahim¹, Zara Hayat², Muhammad Abbas³, Amna Rahim⁴, Muhammad Abdul Rahim⁵

NUST, Islamabad, Pakistan

Email: 1 arahim.cse19ceme@ce.ceme.edu.pk, 2 zhayat.cse19ceme@ce.ceme.edu.pk, 3 abbasamir@hotmail.com

⁴amna1rahim@gmail.com, ⁵marfarooqi@gmail.com,

Abstract—The defect is an error state, which is not according to the requirement of the given software. The error in the software system can affect the efficiency and reliability of the software therefore, early prediction of software defects can save the companies from a bigger loss. For this purpose, multiple algorithms have been proposed in the past decade. Still, the accuracy achieved is not suitable for the market demand which means that a lot of improvement needs to be done. In this paper, we have proposed an efficient and reliable framework for the prediction of the software defects. The framework is divided into three main steps which involves data preprocessing steps that includes: removal of noise and normalization. After that, feature extraction is done using correlation-based analysis and relevant features are selected, and finally applies machine learning models including naïve Bayes and linear regression. Results show that the proposed method can reach an accuracy of 98.7% using Naïve Bayes algorithm. Consequently, the significance of the proposed technique is that it will lower the cost of maintenance and reduce the code complexity by predicting the defects earlier in the software systems that will help the developers to remove those defects and ultimately improves the software quality before the deployment phase.

Keywords— *Software Defect Prediction, PROMISE Dataset, Feature extraction, Naïve Bayes*

I. INTRODUCTION

In a software system, a software defect indicates an error state that does not adhere to the software requirements or the expectations of users. It is an error situation; either in the logic part or in the coding section that produces the program to yield unexpected and typically inaccurate results [1]. Most of the defects arise from the mistakes made by developers or the design architects. In practice, defects of software system include: Arithmetic defects that are caused by any mistake in some arithmetic expressions, Syntax defects are the mistakes caused in the writing style of the code, Logical defects are mistakes in the implementation part of the code, Performance defects causes undesirable results and Interface defects are caused in the interaction of users and the software [2].

Software defects severely threaten the availability and the reliability of software systems. Once the system with errors is implemented, it is very expensive to find and fix the defects in the system. Therefore, prediction of the unknown defects can help the developers and the programmers to find out the important information about the type and the location of defects to improve the inaccuracy of the software system and gain the required confidence level in the system.

Today, prediction of defects in a software system has become one of the most explored topics by the researchers [4].

Predictions of defects help the programmers to locate the defects in the malfunctioning code regions and prioritize their testing techniques by the severity of the defective regions [5]. Defect Prediction allows the developers and software testers to assess the product's quality and check whether the quality standards are met or not and also assesses if the final product meets the users' needs and expectations. Moreover, it eases the distribution of resources for the formal verification of the system to be developed.

There are numerous prediction models and metrics proposed that predicts the defects of software systems. One of the metrics is the size and complexity metrics of a system. Models like, reliability models use the operational usage profile to predict the rate of failures in the system. Multivariate complex statistical models are also introduced to find a single complexity metric that too indicates the defects in the system [3]. Consequently, to guarantee a software system with a high quality, software defect prediction models are proposed that define the relationship between various software defects and the software metrics i.e. Software line of codes and McCabe's Cyclomatic complexity. [4]

It is concluded from the analyses done on defects prediction models that either through manipulating novel features is a better way to epitomize defects manually or by enhancing machine-learning algorithms to improve prediction models to attain preminent results. It can be seen from the research regarding manually designed features that there are various studies, which extricate the faulty files from non-defective ones. Following are several features discussed; Halstead features were established on the counts of operator and operand [6], McCabe features dealt with the dependencies [7], CK features were [8] on the counts of inheritance and function, MOOD features were based on the factors of polymorphism, coupling, etc. [9], code change features were based on the addition or removal of many LOC [10], removed, etc. with several other object-oriented features [11]. With so many features designed manually, there are several machine-learning algorithms utilized i.e. Naïve Bayes for the prediction of software defects, Decision Tree (DT) algorithm, Neural Network (NN) classifier, and Support Vector Machine (SVM) [12].

In this paper, we propose a novel approach for the prediction of software defects by careful feature selection and data handling. The PROMISE dataset is used for this prediction. The proposed paper aimed to predict the software defect with a reduced set of features and improved accuracy. Existing researches have focused on achieving encouraging accuracy with traditional

classification models. However, the proposed solution is focused on improving the overall classification process by handling data noise and applying the feature selection method, followed by the classification methods. Moreover, the performance is tested over different test-train splits to determine the best split. The rest of the paper is arranged as follows: Related work is described in section II; section III presents the materials and methods. Section VI discusses the results. Finally, section VI concludes the paper.

II. RELATED WORK

Several research papers on the prediction of software defects are reviewed. This section summarizes the related research work. The main purpose of software engineers is not only to develop the software as per the functionalities needed but also to provide a high-quality product within the specified requirements that meet the user's satisfaction and their expectations. To predict the quality of the developed system the major focus is to detect and locate the defect-prone regions. To attain the defect prediction strategy, several studies and experimentations are made that introduce methodologies using known training samples with known labels and predict the whole system using supervised learning.

Generally, most of the studies done on the prediction of defects utilized two types of defect data that is mainly primary or secondary, which is taken from real-world software projects [13]. From the discussion above, it is signified that the prediction of software defects has become the researchers most searched domain in the community of software engineering. Therefore, there exists quality rewards and potential cost allied with correctly predicting software with defects [15], [16]. The researchers have concluded from their studies and research that the single and universal classifiers hold no practical value, as it is not enough to generate an accurate prediction model. Therefore, by using local or relevant data, researchers learn and build classifiers. Each of the methodologies has the following fundamental aspects that include; a technique used for learning, which the specific classifier is originated from. Another aspect is the use of defect data to construct and assess the classifier. One aspect is to indicate various kinds of features or input metrics and indicate the accuracy and the performance measures on the prediction done.

Shanthini et al. [13] proposed an ensemble approach of SVM on two datasets where he showed the bagged SVM classifier contributes more performance (through AUC-ROC, Root Mean Square Error (RMSE) and ROC-curves) than the single classifier approach in fault prediction. The author in the paper [14] proved better performance by using Complimentary Naïve Bayes (CNB) and Logistic Model Trees (LMT) models on an intelligent system for the software maintenance severity prediction. Menzies et al. [15] worked on fault prediction by using logNums filter on the NB classifier, which yields the greatest performance. Elish et al. [16] proved SVM provides better performance than statistical models or machine learning models. Kanmani et al. [17] used the dataset of the projects made by graduate students and matched the results of data mining techniques i.e. Backpropagation Neural Network (BPN) with the addition of Probabilistic Neural Network (PNN) and then

compared the results of statistical techniques. He concluded that PNN gave better performance.

III. MATERIALS AND METHODS

This section explains the complete materials and methods used in the proposed paper. Figure 1 shows the block diagram of the proposed scheme.

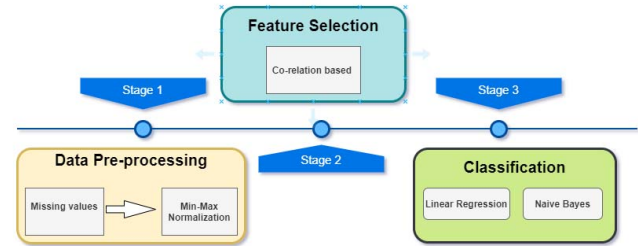


Fig.1 Defect Prediction Framework

A. Dataset

Various datasets are available for the detection of software defects. The dataset, which we have used, is taken from PROMISE Software Engineering Repository [18]. It contains 22 attributes and 10885 samples. In these 22 attributes, 21 are the decision variables and 22nd is the target variable. Figure 2 shows the features of the given dataset.

Attribute Name	Explanation	Type
loc	Line of code	Numeric
V(g)	Cyclomatic Complexity	Numeric
Ev(g)	Essential Complexity	Numeric
Iv(g)	Design Complexity	Numeric
n	Total operators + operands	Numeric
v	Volume	Numeric
L	Program length	Numeric
d	Difficulty	Numeric
i	Intelligence	Numeric
e	Effort	Numeric
b	-	Numeric
t	Time estimator	Numeric
IOCode	Line count	Numeric
IOComment	count of lines of comments	Numeric
IOBlank	count of blank lines	Numeric
IOCodeAndComment	-	Numeric
uniq_op	unique operators	Numeric
uniq_Opnd	Unique operands	Numeric
total_Op	total operators	Numeric
total_Opnd	total operands	Numeric
branchCount	flow graph	Numeric
defects	Reported Defects either true or false	Boolean

Fig.2 Attributes of the dataset

B. Data Preprocessing

Data preprocessing is an important step in machine learning which highly affects the accuracy of the model. Two main steps in data preprocessing are: data cleaning and data transformation [19]. In data cleaning, we will look for the missing values and outliers in the data and will try to remove them using different methodologies that includes; mean substitution or the most probable value. Similarly in data transformation, the data can be converted from one format to another. Normalization, Discretization, and Attribute selection are normally used for this purpose.

In this paper, first we checked the missing values [20] in the data but there were no missing values present in the data so there was no handling of missing values. Secondly, we checked for the outliers in the data using the box plot. The outliers in the data are removed and thus the data is cleaned. Figure 3 shows the

boxplot of the "unique operator" attribute. In the same way, the box plot for all other attributes is made and outliers are detected and then removed. After the data is cleaned, we will move towards the data normalization and make the data in a specific format as needed.

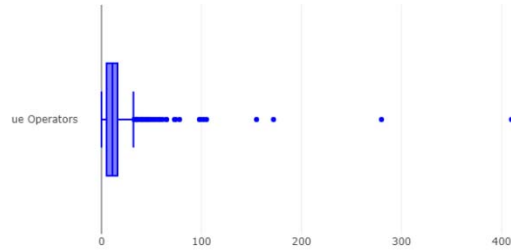


Fig.3 Box Plot of unique operator

In data normalization, we normalized the data which was very high in range as compared to the other samples in the column. To make all the data fall in the specified range we have used min_max normalization. Min-max normalization is one of the most common transformation strategies, which linearly transforms the input data to a new range y . The normalized data is then concatenated with the data which was already in the specified range. The min-max normalization is done using the equation 1 where y is the normalized value, x is the value to be normalized, max is the maximum value of the attribute, min is the minimum value of the attribute, new_{max} is the maximum value of the new range in which we are converting the input and new_{min} is the minimum value of the new range in which we are converting the input.

$$y = \frac{(x-min)}{(max-min)} (new_{max} - new_{min}) + new_{min} \quad (1)$$

C. Feature Selection

The analysis to identify the strongest predictors (feature importance) is best addressed using correlation analysis. This allows it to assess all the features and complete training data to pick the strongest predictors. Other supervised learning approaches that are non-parametric such as K-Nearest Neighbors would not be able to rank the predictors by their importance. We have applied a heat-map analysis using a Spearman correlation matrix of the attributes. It is a visualization method in two-dimension where numerical values are displayed in colors and arranged in rows and columns [The heat-map was coded in Python using the function heat-map from the Seaborn library].

Figure 4 shows the heat-map analysis of the attributes to highlight their correlation. The colors show how one parameter is associated with another parameter through the colors displayed. Lighter colors show that two attributes are highly correlated, white being the most correlated with a value of 1.00. Darker colors, on the other hand, show that two parameters are poorly correlated. In correlation-based feature selection, every attribute is ranked according to the heuristic evaluation function based on correlation of the attributes.

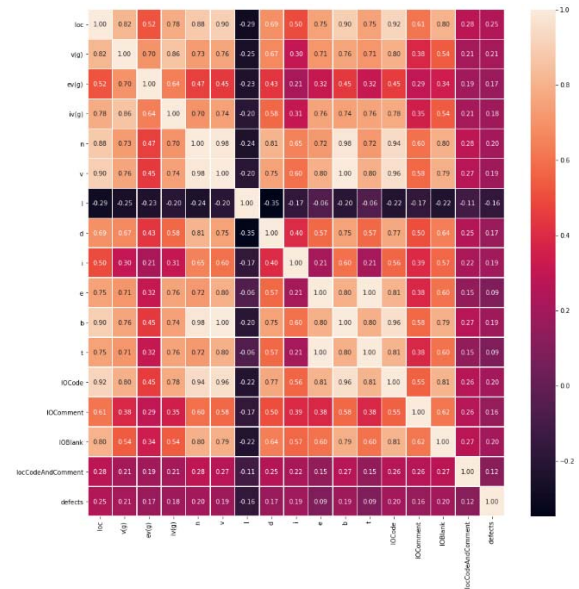


Fig.4 Co-relation based feature selection

The attributes which are selected and their correlation are shown in Table 1.

Table 1 Selected Attributes and their co-relation

	loc	V(g)	Ev(g)	Lv(g)	n	v
loc	1	0.82	0.52	0.78	0.88	0.90
V(g)	0.82	1	0.70	0.86	0.73	0.76
Ev(g)	0.52	0.70	1	0.64	0.47	0.45
Lv(g)	0.78	0.86	0.64	1	0.70	0.74
n	0.88	0.73	0.47	0.70	1	0.98
v	0.90	0.76	0.45	0.74	0.98	1

D. Classification

The algorithms which were used for classification of the machine learning models includes Naïve Bayes and Linear Regression. These algorithms were used to determine the performance with a reduced feature space. Their performance and accuracies are analyzed, followed by cross-validation techniques.

Naïve is based on the Bayes theorem which is a statistical classification technique. It is one of the efficient algorithms which gives good accuracy on large datasets. It makes the assumption that features are independent of each other which is not true in the real world. This assumption is known as the class conditional probability given in equation 2. $P(h)$ is known as the prior probability, $P(D)$ is the probability of the data, $P(D|h)$ is the posterior probability which tells about the probability of the data given the hypothesis h . It is the simplest algorithm that can be extended for multiple classes as well.

$$P(h|D) = \frac{P(D|h)P(h)}{P(D)} \quad (2)$$

Linear Regression is a predictive model used for regression analysis. It models the relationship between two variables while fitting an equation to the data. In these two variables, one is known as the explanatory variable and the other is the

dependent variable. Equation 3 is used for fitting the equation. Table 2 gives the accuracies achieved using these models.

$$Y = bX + a \quad (3)$$

TABLE 2 COMPARISON OF MODELS

Machine Learning Method	Ratio (training: testing) 80:20
	Accuracy
Linear Regression	95.7%
Naïve Bayes	98.7%

IV. DISCUSSION AND RESULTS

Prediction of software defects in the phase of software testing and maintenance has attracted a lot of IT organizations as well as the researcher's community to identify the necessities for the improvement and success of software projects. The main differentiator in a competitive market-place is the customer satisfaction that can be achieved by providing reliable products. Therefore, accuracy is considered very vital for the reliable prediction of software defects. Existing researches have focused on achieving improved accuracy with traditional classification models. However, the proposed solution is focused on improving the overall classification process by handling data noise and applying the feature selection method, followed by the classification methods. Moreover, the performance is tested over different test-train splits to determine the best split.

In the proposed methodology firstly, we have performed data preprocessing which includes: 1) Removing outliers using box plot 2) Handling missing values using mean substitution 3) Min-Max normalization to make all the features fall in the specified range. After that, feature selection was done in which features that were highly correlated were selected. This resulted in a reduced feature set of 17 features. Finally, we applied machine learning models and achieved an accuracy up to 98.7% by using Naïve Bayes classifier and 95.7% using Linear Regression. Consequently, best accuracy was achieved by applying Naïve Bayes algorithm.

Table 2 epitomizes various machine learning models proposed by different authors along with their achievements. It can be evidently seen through the comparison, that our results provide far better performance than others. In this way the overall process is improved by increasing the accuracy and reducing the computation time as less number of features are used.

TABLE 3 COMPARISON OF WORK OF DIFFERENT AUTHORS

Author	feature	Model	Validation	Accuracy
Shanthini [13]	All features	Bagged SVM	10-fold cross validation	84%
Sandhu [14]	All features	Simple Logistic	10-fold cross validation	86%
Elish [16]	CFS	SVM	-	93%
Kanmani [17]	PCA	PNN	-	95.63%
Our Approach	Co-relation based	Naïve Bayes	10-fold cross validation	98.7%

V. CONCLUSION AND FUTURE WORK

Machine learning techniques have been applied extensively over the years in a variety of software fields for detection and prediction and various defects and errors. They are considered

as well-thought-out valuable techniques that will be able to guide software engineers when it comes to making decisions on data that is accessible as well as aiding in the development of new software.

Our proposed solution was a machine learning-based solution for the prediction of software defects. We did data preprocessing by removing the outliers from the data and did normalization to make the data fall in the specified range. After that correlation-based feature selection is used for the feature extraction. Finally, naïve Bayes and linear regression are used for training the model and an accuracy of 98%, and 95% have been achieved from them respectively. In the future, the same framework can be implemented on the other datasets and their accuracies can be analyzed on them as well.

REFERENCES

- [1] "Defect - Software Testing Fundamentals", Software Testing Fundamentals, 2020. [Online]. Available: <http://softwaretestingfundamentals.com/defect/>. [Accessed: 19- May- 2020].
- [2] "Types of Defects in Software Development - GeeksforGeeks", *GeeksforGeeks*, 2020. [Online]. Available: <https://www.geeksforgeeks.org/types-of-defects-in-software-development/>. [Accessed: 19- May- 2020].
- [3] Fenton, Norman E., and Martin Neil. "A critique of software defect prediction models." *IEEE Transactions on software engineering* 25, no. 5 (1999): 675-689.
- [4] Moser, Raimund, Witold Pedrycz, and Giancarlo Succi. "A comparative analysis of the efficiency of change metrics and static code attributes for defect prediction." In *Proceedings of the 30th international conference on Software engineering*, pp. 181-190. 2008.
- [5] Wang, Song, Taiyue Liu, and Lin Tan. "Automatically learning semantic features for defect prediction." In *2016 IEEE/ACM 38th International Conference on Software Engineering (ICSE)*, pp. 297-308. IEEE, 2016.
- [6] M. H. Halstead. *Elements of Software Science* (Operating and programming systems series). Elsevier Science Inc., 1977.
- [7] T. J. McCabe. A complexity measure. *TSE'76*, (4):308-320.
- [8] S. R. Chidamber and C. F. Kemerer. A Metrics Suite for Object-Oriented Design. *TSE'94*, 20(6):476-493.
- [9] R. Harrison, S. J. Counsell, and R. V. Nithi. An evaluation of the mood set of object-oriented software metrics. *TSE'98*, 24(6):491-496.
- [10] T. Jiang, L. Tan, and S. Kim. Personalized defect prediction. In *ASE'13*, pages 279-289.
- [11] F. B. e Abreu and R. Carapu,ca. Candidate metrics for object-oriented software within a taxonomy framework. *JSS'94*, 26(1):87-96.
- [12] X.-Y. Jing, S. Ying, Z.-W. Zhang, S.-S. Wu, and J. Liu. Dictionary learning-based software defect prediction. In *ICSE'14*, pages 414-423.
- [13] Shanthini, A., and R. M. Chandrasekaran. "Analyzing the effect of bagged ensemble approach for software fault prediction in class level and package level metrics." In *International Conference on Information Communication and Embedded Systems (ICICES2014)*, pp. 1-5. IEEE, 2014.
- [14] Sandhu, Parvinder Singh, Sunil Kumar, and Hardeep Singh, "Intelligence System for Software Maintenance Severity Prediction", *Journal of Computer Science*, Vol. 3 (5), pp. 281-288, 2007.
- [15] Menzies, T., Greenwald, J., & Frank, A. Data mining static code attributes to learn defect predictors. *IEEE Transactions on Software Engineering*, 33(1), 2007.
- [16] K.O. Elish, M.O. Elish, Predicting defect-prone software modules using support vector machines, *Journal of Systems and Software* 81 (5) 649-660, 2008.
- [17] S.Kanmani, V.R. Uthariaraj, V.Sankaranarayanan, P.Thambidurai, "Objected-oriented software fault prediction using neural networks", *Information and Software Technology* 49 (5) 483 - 492, 2007.

- [18] PROMISE JM1:
<http://promise.site.uottawa.ca/SERepository/datasets/jm1.arff>
- [19] D. Ferreira, H. Peixoto, J. Machado and A. Abelha, "Predictive Data Mining in Nutrition Therapy," 2018 13th APCA International Conference on Automatic Control and Soft Computing (CONTROLO), Ponta Delgada, 2018, pp. 137-142, doi: 10.1109/CONTROLO.2018.8516413.
- [20] S. M. Abu-Sound, "A Novel Approach for Dealing with Missing Values in Machine Learning Datasets with Discrete Values," 2019 International Conference on Computer and Information Sciences (ICCIS), Sakaka, Saudi Arabia, 2019, pp. 1-5, DOI: 10.1109/ICCISci.2019.8716430.